



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

N° de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 10 de octubre de 2026

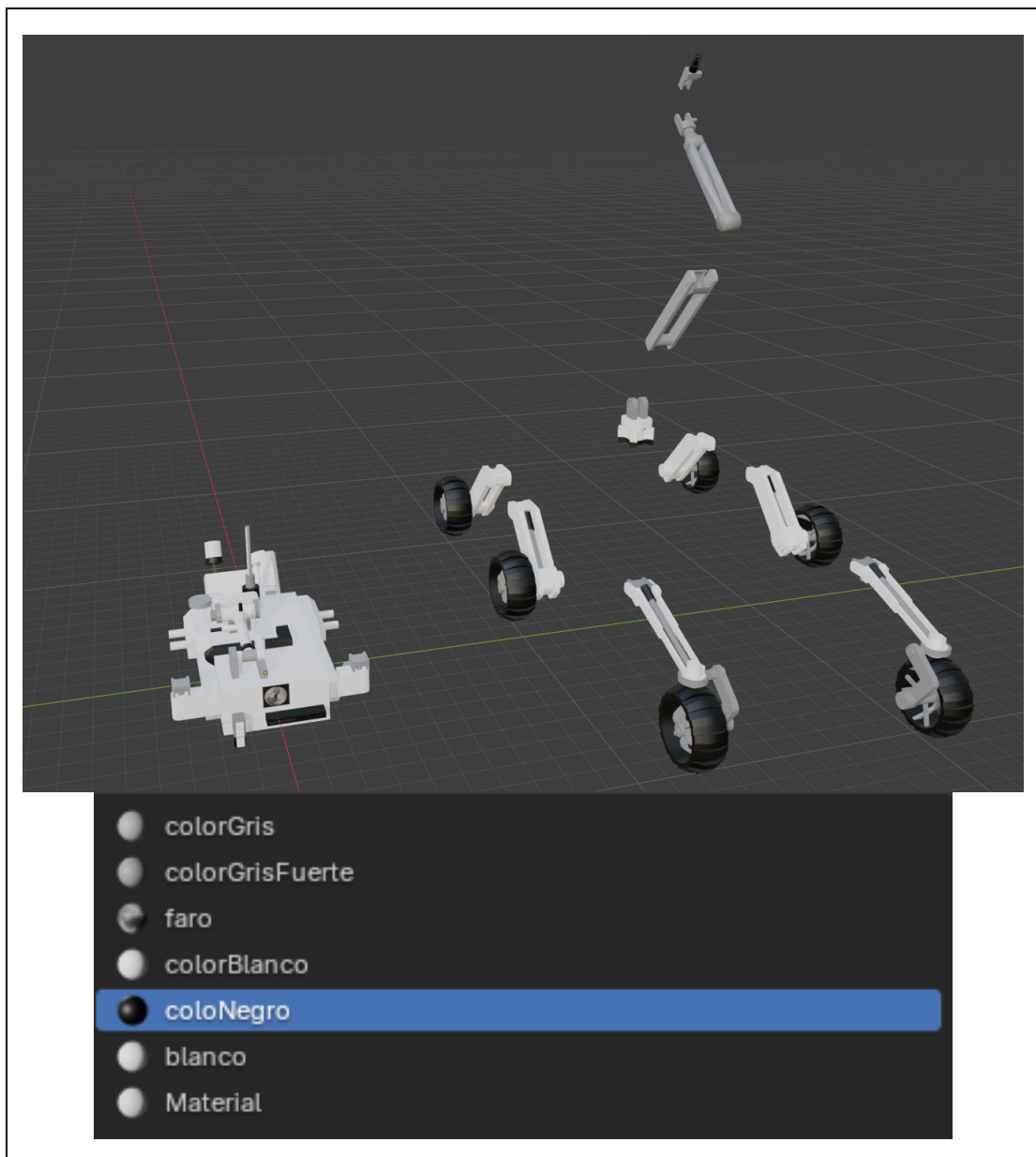
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.1 Añadir una luz de faro (spotlight) frontal al rover con jerarquía suyo que ilumine hacia adelante y al avanzar o retroceder el Rover, la luz siga unida a él.

Primero en Blender edité todas las partes del modelo del rover para agregarles textura con imágenes, para que así ya tuviera un color definido cada parte del rover:



Después exporté los modelos y en *P07_320152896.cpp* modifiqué los bloques de código siguientes:

- Carga de modelos:

```
57 //Lista de Modelos a importar
58 //Rover
59 Model baseRover;
60 Model pinza;
61 Model brazo1;
62 Model brazo2;
63 Model baseBrazo;
64 Model llantaDelanteraCerca;
65 Model llantaDelanteraLejos;
66 Model llantaMedioCerca;
67 Model llantaMedioLejos;
68 Model llantaAtrasCerca;
69 Model llantaAtrasLejos;
```

```
446 //Rover
447 baseRover = Model();
448 baseRover.LoadModel("Models/baseRover.fbx");
449
450 baseBrazo = Model();
451 baseBrazo.LoadModel("Models/baseBrazo.fbx");
452
453 brazo1 = Model();
454 brazo1.LoadModel("Models/brazo1.fbx");
455
456 brazo2 = Model();
457 brazo2.LoadModel("Models/brazo2.fbx");
458
459 pinza = Model();
460 pinza.LoadModel("Models/pinza.fbx");
461
462 llantaDelanteraCerca = Model();
463 llantaDelanteraCerca.LoadModel("Models/llantaDelanteraCerca.fbx");
464
465 llantaDelanteraLejos = Model();
466 llantaDelanteraLejos.LoadModel("Models/llantaDelanteraLejos.fbx");
467
468 llantaMedioCerca = Model();
469 llantaMedioCerca.LoadModel("Models/llantaMedioCerca.fbx");
470
471 llantaMedioLejos = Model();
472 llantaMedioLejos.LoadModel("Models/llantaMedioLejos.fbx");
473
474 llantaAtrasCerca = Model();
475 llantaAtrasCerca.LoadModel("Models/llantaAtrasCerca.fbx");
476
477 llantaAtrasLejos = Model();
478 llantaAtrasLejos.LoadModel("Models/llantaAtrasLejos.fbx");
479
```

➤ Dibujo del rover:

```
628 //Rover
629 //Modelo Inicial
630 model = glm::mat4(1.0f);
631 color = glm::vec3(1.0f, 1.0f, 1.0f);
632 model = glm::translate(model, glm::vec3(1.0f + mainWindow.getmuevex(), -1.0f, 6.0));
633 model = glm::translate(model, glm::vec3(0.0f, 2.8f, 10.0f));
634 modelaux = model;
635 modelaux2 = model;
636 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
637 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
638 baseRover.RenderModel(); //modificar por el modelo de solo cuerpo del Rover,
639 //para que se pueda separar el brazo y las llantas
640 //color = glm::vec3(0.0f, 0.0f, 1.0f);
641 //En sesión se separara una parte del modelo y se unirá por jeraquia al cuerpo
642
643 // ----- Brazo -----
644 // Base del brazo
645 model = modelaux;
646 model = glm::translate(model, glm::vec3(1.75f, 1.5f, -1.0f));
647 modelaux = model;
648 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
649 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
650 baseBrazo.RenderModel();
651
652 // Brazo 1 (conectado a la base del brazo)
653 model = modelaux;
654 model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f));
655 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
656 modelaux = model;
657 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
658 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
659 brazo1.RenderModel();
660
661 // Brazo 2 (conectado al brazo 1)
662 model = modelaux;
663 model = glm::translate(model, glm::vec3(2.45f, 2.5f, -0.01f));
664 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
665 modelaux = model;
666 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
667 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
668 brazo2.RenderModel();
669
670 // Pinza (conectado al brazo 2)
671 model = modelaux;
672 model = glm::translate(model, glm::vec3(-3.4f, 3.3f, 0.125f));
673 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
674 modelaux = model;
675 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
676 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
677 pinza.RenderModel();
678
679 // Llanta delantera derecha
680 model = modelaux2;
681 model = glm::translate(model, glm::vec3(2.075f, 1.15f, 3.07f));
682 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
683 modelaux = model;
684 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
685 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
686 llantaDelanteraLejos.RenderModel();
687
688 // Llanta delantera izquierda
689 model = modelaux2;
690 model = glm::translate(model, glm::vec3(2.075f, 1.14f, -3.05f));
691 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
692 modelaux = model;
693 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
694 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
695 llantaDelanteraCerca.RenderModel();
696
697 //Llanta media derecha
698 model = modelaux2;
699 model = glm::translate(model, glm::vec3(-2.5f, 0.25f, 3.07f));
700 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
701 modelaux = model;
702 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
703 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
704 llantaMedioLejos.RenderModel();
705
706 // Llanta media izquierda
707 model = modelaux2;
708 model = glm::translate(model, glm::vec3(-2.45f, 0.25f, -3.07f));
709 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
710 modelaux = model;
711 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
712 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
713 llantaMedioCerca.RenderModel();
```

```

715 //Llanta trasera derecha
716 model = modelaux2;
717 model = glm::translate(model, glm::vec3(-3.5f, 0.25f, 3.07f));
718 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
719 modelaux = model;
720 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
721 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
722 llantaAtrasLejos.RenderModel();
723
724 // Llanta trasera izquierda
725 model = modelaux2;
726 model = glm::translate(model, glm::vec3(-3.55f, 0.25f, -3.07f));
727 //model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
728 modelaux = model;
729 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
730 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
731 llantaAtrasCerca.RenderModel();

```

➤ Luz del faro:

```

550 // 1. Faro Frontal Rover (Índice 1)
551 spotLights[1] = SpotLight(1.0f, 1.0f, 1.0f,
552 1.0f, 1.0f,
553 0.0f, 2.8f, 0.0f,
554 1.0f, 0.0f, 0.0f, // Dirección hacia adelante (+X)
555 1.0f, 0.0f, 0.0f,
556 10.0f); // Apertura del faro
557 spotLightCount++;

```

```

587 // Matrices y posiciones de las luces para la jerarquía
588 glm::mat4 roverMatrix(1.0f);
589 glm::mat4 avionMatrix(1.0f);
590
591 glm::vec3 roverPos;
592 glm::vec3 roverDir;

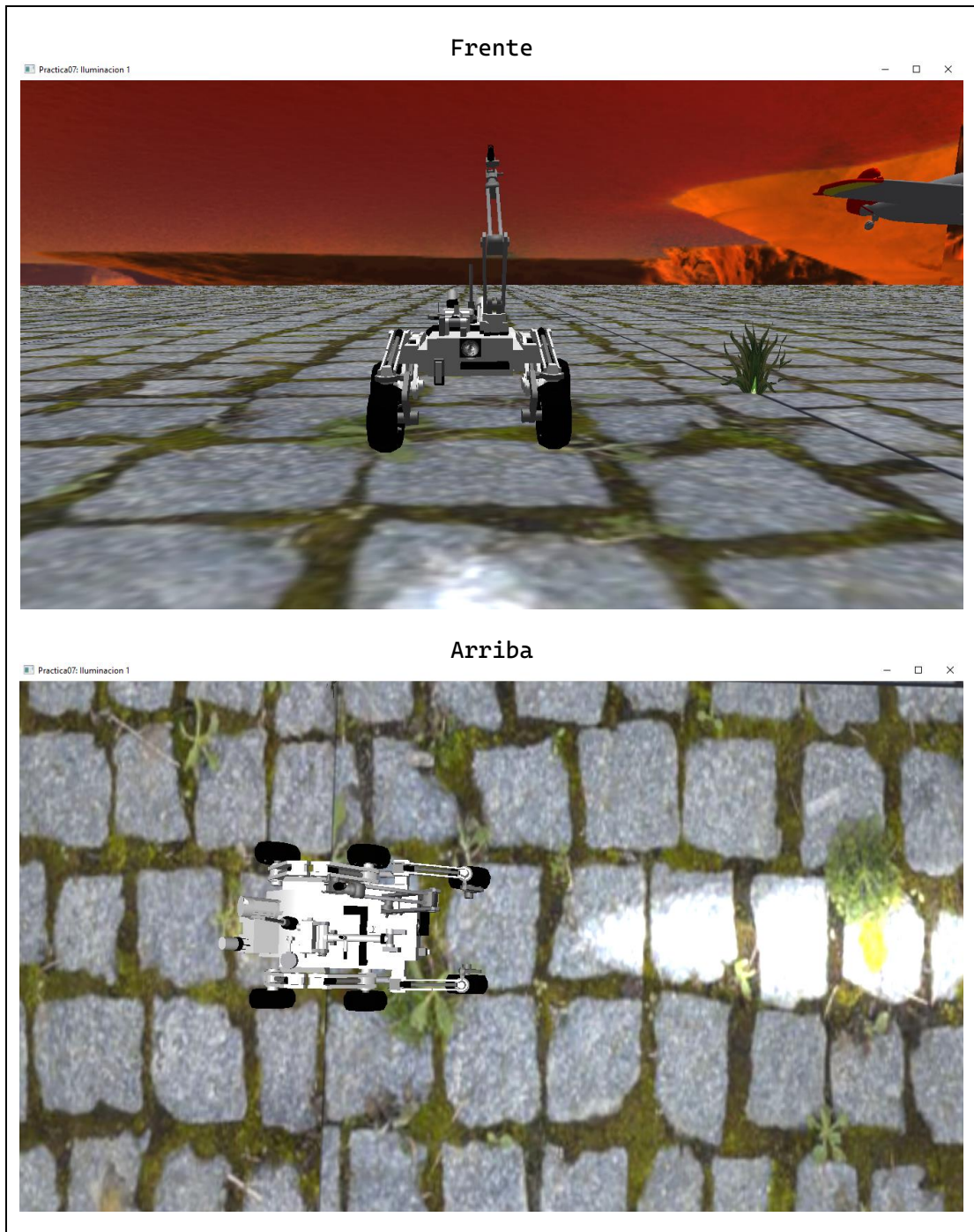
```

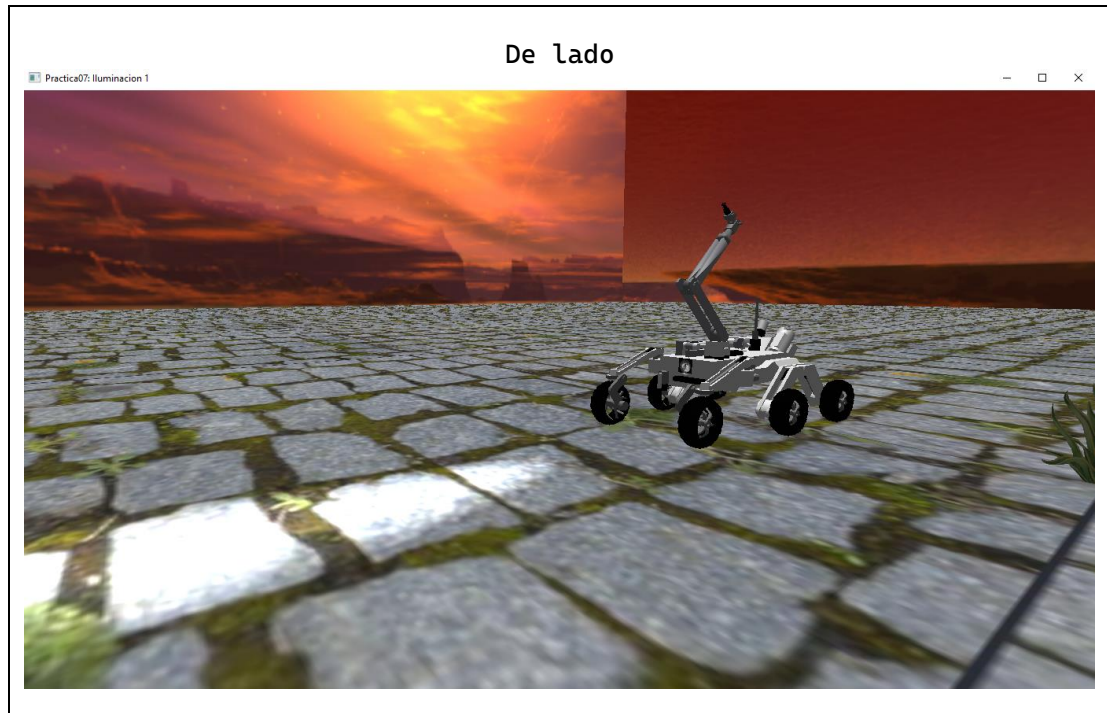
```

672 //Rover
673 //Modelo Inicial
674 model = glm::mat4(1.0);
675 color = glm::vec3(1.0f, 1.0f, 1.0f);
676 model = glm::translate(model, glm::vec3(1.0f + mainWindow.getmuevex(), -1.0f, 6.0));
677 model = glm::translate(model, glm::vec3(0.0f, 2.8f, 10.0f));
678 modelaux = model;
679 modelaux2 = model;
680
681 // Matriz del rover para calcular la posición y dirección del faro
682 roverMatrix = modelaux2;
683
684 // Posición del faro del rover
685 roverPos = glm::vec3(roverMatrix * glm::vec4(1.0f, 1.0f, 0.0f, 1.0f));
686
687 // Dirección del faro del rover
688 roverDir = glm::normalize(glm::vec3(roverMatrix * glm::vec4(1.5f, -0.25f, 0.0f, 0.0f)));
689
690 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
691 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
692 baseRover.RenderModel();
693
694 // Faro del rover
695 spotLights[1].SetFlash(roverPos, roverDir);

```

Agrego capturas de pantalla del rover:





NOTA. Para las capturas de pantalla desactivé la luz de “linterna” para que sólo se vea la luz del faro, en el video aparece esta luz activada.

1.2 Añadir una luz de faro (spotlight) al avión con jerarquía suya que ilumine hacia abajo y al subir y bajar el avión, la luz siga unida a él.

Primero en Blender agregué y edité una textura para dos faros:



Después exporté el modelo y en *P07_320152896.cpp* modifiqué los bloques de código siguientes:

➤ Carga de modelo:

```
70 //Añadir modelo del avión.
71 Model Avion_M;

480 //Añadir modelo del avión
481 Avion_M = Model();
482 Avion_M.LoadModel("Models/avioncito.obj");
483
```

➤ Dibujo del avioncito:

```
752 //Añadir modelo del avión
753 model = glm::mat4(1.0);
754 model = glm::translate(model, glm::vec3(1.0f + mainWindow.getmuevexAv1(), -1.0f, 6.0));
755 model = glm::translate(model, glm::vec3(1.0f, 1.0f + mainWindow.getmuevexAv2(), 6.0));
756 model = glm::translate(model, glm::vec3(0.0f, 10.0f, -25.0));
757 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
758 //model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
759 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
760 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
761 color = glm::vec3(1.0f, 1.0f, 1.0f);
762 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
763 Avion_M.RenderModel();
```

➤ Luz del faro hacia abajo:

```
559 // 2. Faro hacia abajo del avioncito (índice 2)
560 spotLights[2] = SpotLight(1.0f, 1.0f, 0.8f, // Color Blanco un poco más cálido para diferenciar la tonalidad
561 1.0f, 1.0f,
562 0.0f, 0.0f, 0.0f,
563 0.0f, -1.0f, 0.0f, // Dirección hacia abajo (-Y)
564 1.0f, 0.0f, 0.0f,
565 15.0f);
566 spotLightCount++;
```

```
587 // Matrices y posiciones de las luces para la jerarquía
588 glm::mat4 roverMatrix(1.0f);
589 glm::mat4 avionMatrix(1.0f);

590 glm::vec3 roverPos;
591 glm::vec3 roverDir;

592 glm::vec3 avionAbajoPos;
593 glm::vec3 avionAbajoDir;
```

```

806 //Añadir modelo del avión
807 model = glm::mat4(1.0f);
808 model = glm::translate(model, glm::vec3(1.0f + mainWindow.getmuevexAv1(), -1.0f, 6.0f));
809 model = glm::translate(model, glm::vec3(1.0f, 1.0f + mainWindow.getmuevexAv2(), 6.0f));
810 model = glm::translate(model, glm::vec3(0.0f, 10.0f, -25.0f));
811
812 // Matriz del rover para calcular la posición y dirección de los faros
813 avionMatrix = model;
814 avionMatrix = glm::rotate(avionMatrix, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
815
816 // Faro hacia abajo del avión (índice 2)
817 avionAbajoPos = glm::vec3(avionMatrix * glm::vec4(-0.5f, -0.25f, 6.5f, 1.0f));
818 avionAbajoDir = glm::normalize(glm::vec3(avionMatrix * glm::vec4(0.0f, -1.0f, 0.0f, 0.0f)));
819 spotLights[2].SetFlash(avionAbajoPos, avionAbajoDir);
820
821 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
822 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
823 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
824 color = glm::vec3(1.0f, 1.0f, 1.0f);
825 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
826 Avion_M.RenderModel();

```

En *Window.cpp* y *Window.h* modifiqué los bloques de código siguientes:

- *Window.cpp*: Para mover hacia adelante/atrás se asignaron las teclas **M/N**, mientras que para mover arriba/abajo se asignaron las teclas **Z/X**

```

17 muevexAv1= 0.0f;
18 muevexAv2 = 0.0f;
117 if (key == GLFW_KEY_M)
118 {
119     theWindow->muevexAv1 += 1.0;
120 }
121 if (key == GLFW_KEY_N)
122 {
123     theWindow->muevexAv1 -= 1.0;
124 }
125 if (key == GLFW_KEY_X)
126 {
127     theWindow->muevexAv2 += 1.0;
128 }
129 if (key == GLFW_KEY_Z)
130 {
131     theWindow->muevexAv2 -= 1.0;
132 }

```

- *Window.h*:

```

17 GLfloat getmuevexAv1() { return muevexAv1; }
18 GLfloat getmuevexAv2() { return muevexAv2; }

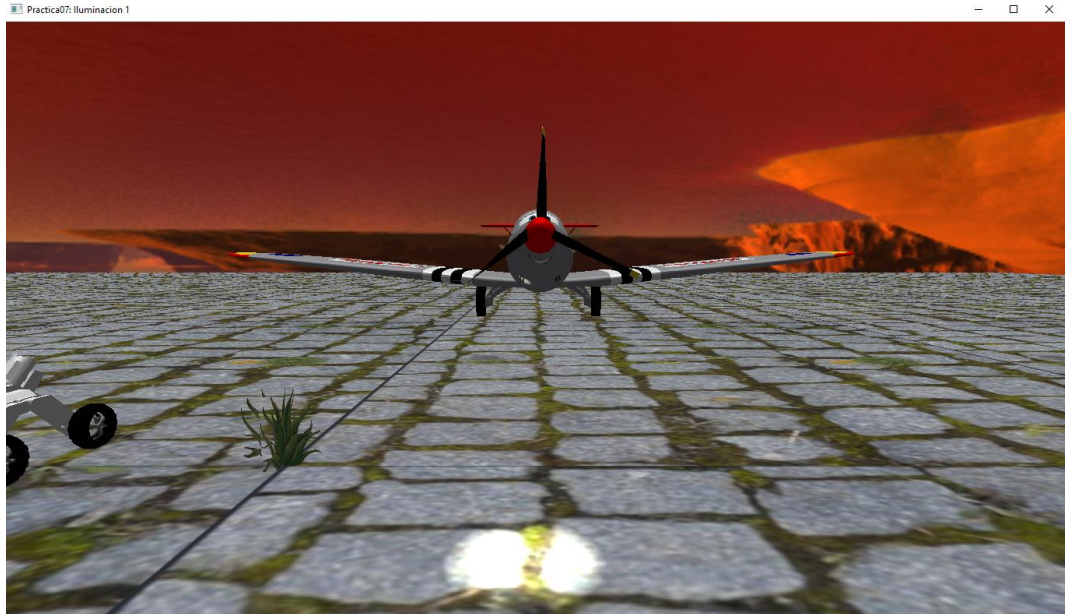
```

```
36  
37
```

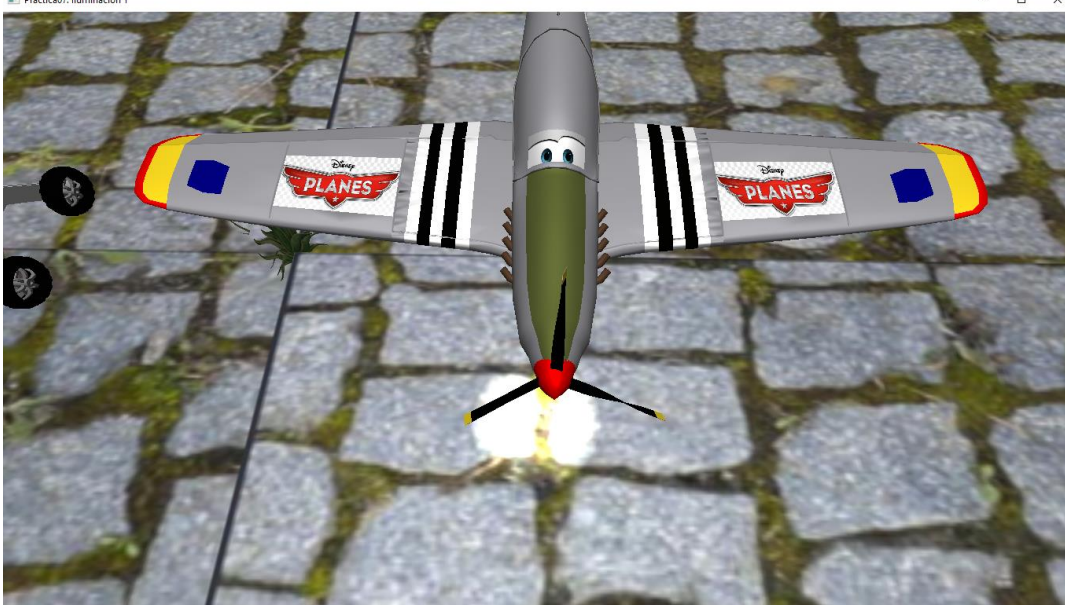
```
GLfloat muevexAv1;  
GLfloat muevexAv2;
```

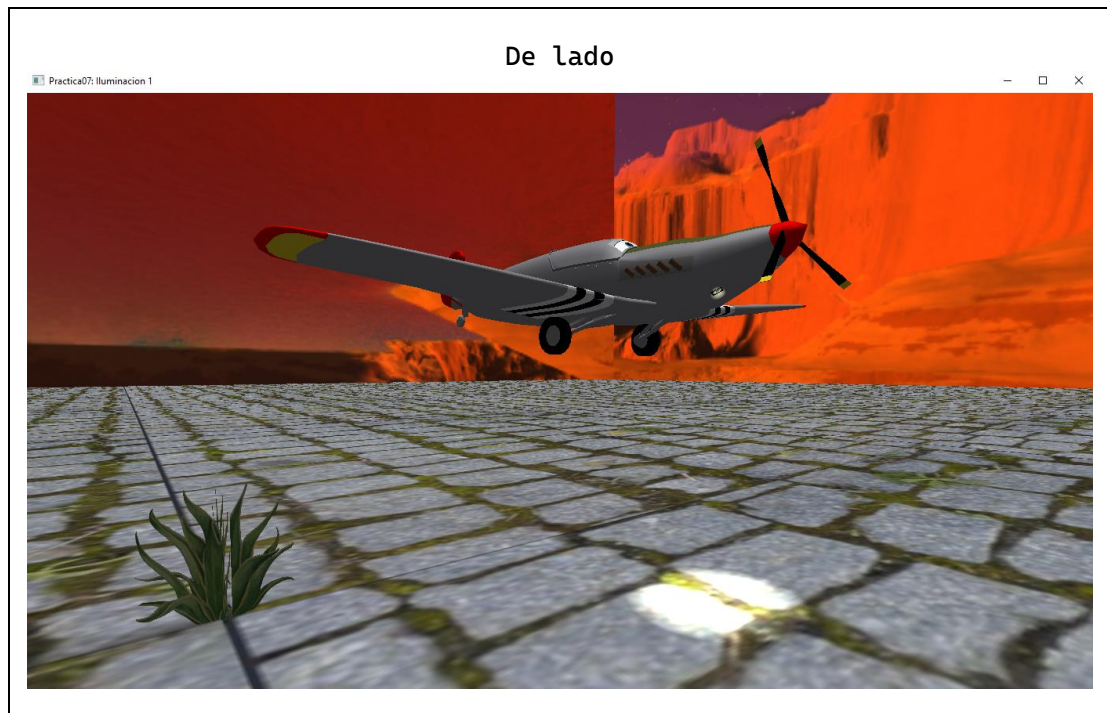
Agrego capturas de pantalla del avioncito:

Frente



Arriba





NOTA. Para las capturas de pantalla desactivé la luz de “linterna” para que sólo se vea la luz del faro, en el video aparece esta luz activada.

1.3 Añadir una luz de faro (spotlight) al avión con jerarquía suya que ilumine hacia adelante y al avanzar o retroceder el avión, la luz siga unida a él.

En `P07_320152896.cpp` modifiqué los bloques de código siguientes:

➤ Luz del faro frontal:

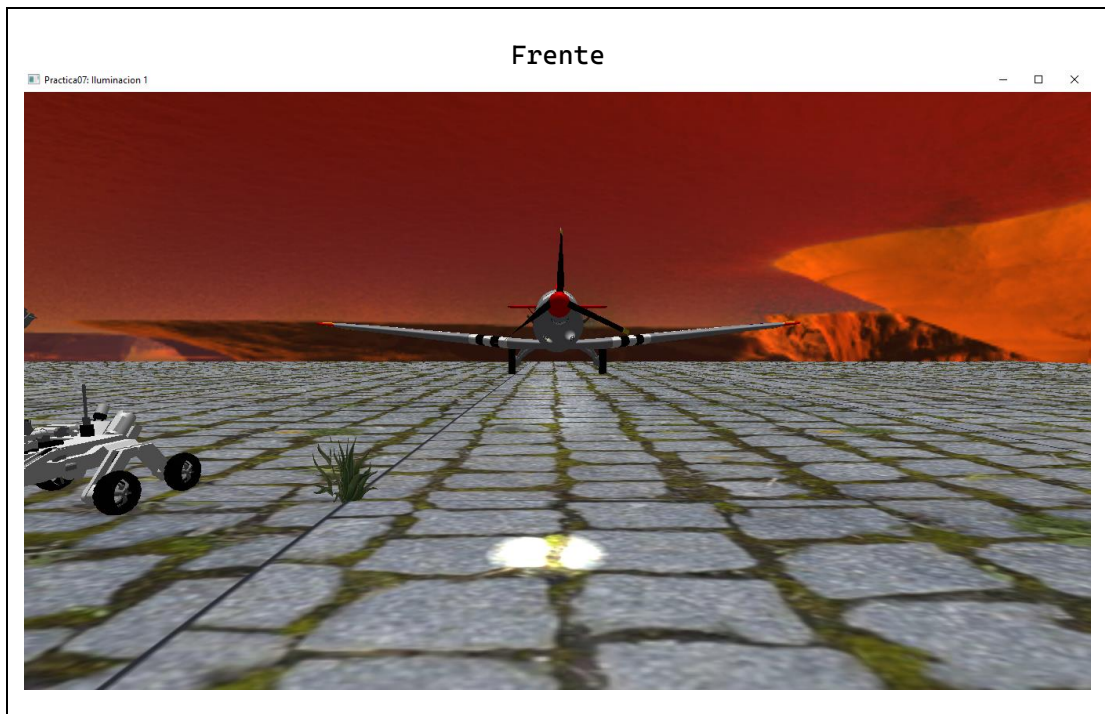
```
568 // 3. Faro frontal del avioncito (Índice 3)
569 spotLights[3] = SpotLight(1.0f, 1.0f, 1.0f,
570     1.0f, 1.0f,
571     0.0f, 0.0f, 0.0f,
572     1.0f, 0.0f, 0.0f, // Dirección hacia adelante (+X)
573     1.0f, 0.0f, 0.0f,
574     20.0f);
575 spotLightCount++;
```

```

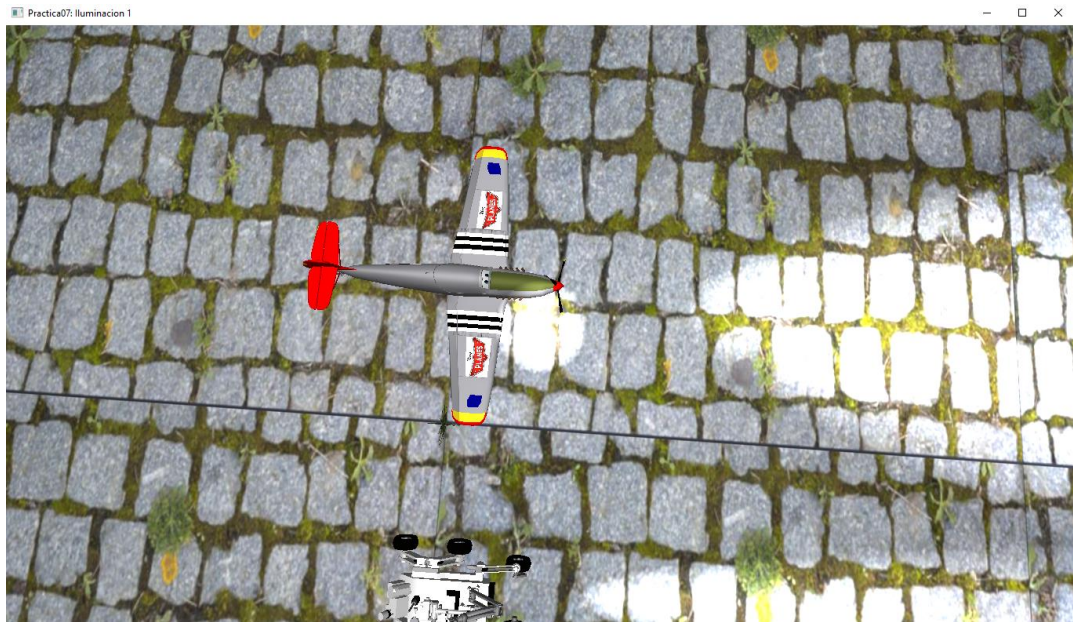
806 //Añadir modelo del avión
807 model = glm::mat4(1.0);
808 model = glm::translate(model, glm::vec3(1.0f + mainWindow.getmuevexAv1(), -1.0f, 6.0));
809 model = glm::translate(model, glm::vec3(1.0f, 1.0f + mainWindow.getmuevexAv2(), 6.0));
810 model = glm::translate(model, glm::vec3(0.0f, 10.0f, -25.0));
811
812 // Matriz del rover para calcular la posición y dirección de los faros
813 avionMatrix = model;
814 avionMatrix = glm::rotate(avionMatrix, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
815
816 // Faro hacia abajo del avión (índice 2)
817 avionAbajoPos = glm::vec3(avionMatrix * glm::vec4(-0.5f, -0.25f, 6.5f, 1.0f));
818 avionAbajoDir = glm::normalize(glm::vec3(avionMatrix * glm::vec4(0.0f, -1.0f, 0.0f, 0.0f)));
819 spotLights[2].SetFlash(avionAbajoPos, avionAbajoDir);
820
821 // Faro frontal del avión (índice 3)
822 avionFrentPos = glm::vec3(avionMatrix * glm::vec4(0.6f, -0.95f, 6.0f, 1.0f));
823 avionFrentDir = glm::normalize(glm::vec3(avionMatrix * glm::vec4(0.0f, -0.2f, 1.0f, 0.0f)));
824 spotLights[3].SetFlash(avionFrentPos, avionFrentDir);
825
826 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
827 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
828 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
829 color = glm::vec3(1.0f, 1.0f, 1.0f);
830 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
831 Avion_M.RenderModel();

```

Agrego capturas de pantalla del avioncito:



Arriba



De lado



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no se presentaron problemas.

3. Conclusión:

- a. Los ejercicios del reporte:** Me gustó mucho el practicar el la iluminación y seguir aplicando el texturizado a los modelos con los que trabajamos previamente.
- b. Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado al texturizado e iluminación.
- c. Conclusión.** Esta práctica me sirvió mucho para saber cómo funciona la iluminación en OpenGL. Al principio se me hizo bastante complicado entender conceptualmente el funcionamiento de la iluminación, sin embargo, a través de la práctica y apoyándome con *Google Gemini*, logré una correcta implementación de las luces solicitadas. Finalmente, al ser el último código “nuevo” que veremos, ya me puedo ir haciendo una idea de cómo ir implementando algunos elementos para mi proyecto final,

Referencias bibliográficas.

<https://stackoverflow.com/questions/17327906/what-glmnormalize-does>